

m-format constellation — Command-line Examples

mnemonic-toolkit v0.55.3 — worked examples (Linux)

2026-06-14

Contents

About these examples	2
1. Install the constellation on Linux	3
2. Single-sig card set from a seed phrase	4
3. Conventional 2-of-3 multisig card set from 3 seed phrases	5
4. Card set -> Bitcoin Core wallet descriptor (and how to import)	6
5. Custom degrading-miniscript wallet (distinct keys per tier) + Nunchuk export	7
5.1 Translate the policy to miniscript	7
5.2 The guided builder caps complexity – use the raw descriptor path	7
5.3 Engrave the card set	8
5.4 Watch-only export for Nunchuk (and other wallets)	8
6. Taproot version of the degrading wallet	9
6.1 Why depth-1 (the one-tier-per-leaf limit)	9
6.2 Build + validate	9
6.3 Engrave the card set + first address	9
6.4 Export for wallets (Nunchuk / Core / Sparrow)	10
6.5 Adding a condition: which depth? (a spending-cost comparison)	10
Appendix — the public test seeds used	12

About these examples

This document shows real, copy-pasteable command lines for the **m-format constellation** — a steel-engravable Bitcoin backup system built around four CLIs (`mnemonic`, `md`, `ms`, `mk`) plus the `mnemonic-gui` overlay. Every command below was executed against `mnemonic v0.55.3` on Linux and the output is reproduced verbatim (long descriptors wrap with a grey hook-arrow continuation marker in the margin).

All seed phrases here are public BIP-39 TEST VECTORS (e.g. `abandon abandon ... about`). They are world-known and hold no funds. **Never type a real seed phrase onto the command line of a networked machine, and never reuse these test phrases for real money.** The toolkit itself is alpha software — use only with disposable amounts or on testnet until it has been independently audited.

The three engraved cards:

Card	What it carries
ms1	BIP-39 entropy (recovers the seed)
mk1	xpub + origin (master fingerprint + BIP path)
md1	wallet policy (descriptor template + bound xpubs)

Throughout, `$` is the shell prompt; everything after it is what you type.

1. Install the constellation on Linux

The in-repo installer builds every component with `cargo install --locked` into `~/.cargo/bin` (no `sudo`, no system files touched). It requires `cargo`, `git`, and a C toolchain. The CLIs need `rustc >= 1.85`; the `mnemonic-gui` overlay needs `rustc >= 1.88`.

Install all five components (4 CLIs + GUI):

```
$ sh -c "$(curl -fsSL https://raw.githubusercontent.com/bg002h/mnemonic-toolkit/master/scripts/install.sh)"
```

Install only the `mnemonic` CLI (the installer carries the current version pin, so it never goes stale):

```
$ sh -c "$(curl -fsSL https://raw.githubusercontent.com/bg002h/mnemonic-toolkit/master/scripts/install.sh)" -- --only  
  ↪ mnemonic
```

Useful installer flags (`scripts/install.sh --help`): `--only <c>`, `--exclude <c>`, `--no-gui` (skip the GUI if your `rustc` is older than 1.88), `--dry-run`, `--list`, `--force`.

Verify:

```
$ mnemonic --version  
$ mnemonic --help      # lists every subcommand
```

2. Single-sig card set from a seed phrase

Create a native-segwit (BIP-84, `m/84'/0'/0'`) single-sig 3-card bundle from one seed phrase. **Best practice: pass the phrase on stdin** (`--slot @0.phrase=-`), not on the command line, so it never lands in `/proc/$PID/cmdline` or your shell history:

```
$ printf '%s' "abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon about" \
  | mnemonic bundle --template bip84 --slot @0.phrase=- --network mainnet
```

(`--template` choices: `bip44`, `bip49`, `bip84`, `bip86` for single-sig.)

Output — the three cards to engrave (stdout):

```
# ms1 (entropy, BCH-checksummed)
ms10entrsqqqqqqqqqqqqqqqqqqqqqqqqqqqqc9sxxraq34v7f

# mk1 (xpub + origin)
mk1qprsqhpbqqsq3cqtstleoutks2qvzg3vs70mejkhk622ws2kgdemj2cd8zwj2skzx2wq0qw7014q99vdyh5x0z8v4yslsp8qp3yxg3dpe854wq4
mk1qprsqhpb0f30mtxzd65mvwcur9usdatwuqvq6z70r9nwrqk6xn6l8gy6nwa2n977sw6zh34rma0nh

# md1 (wallet policy)
md1fgdxlpqpqpm6jzzqvvqpdqw0za5zs4gyy55aq4vsmnhy4s6wyaypu34c7raqu8np
md1fgdxlpqf2zcgefcpupmel75q5435j7seugaj5jr7qyur6vt76es5cdeyrq7zdy0d
md1fgdxlpq3xa2dk8vwpj7gx74hwqxqdp083jehp5tdrfa0n5zdfkqcdlrvnh5r62jn
```

A human-readable **engraving-card panel** is also printed on stderr (suppress it with `--no-engraving-card`):

```
# === Wallet bundle: bip84, mainnet ===
# fingerprint: 73c5da0a
# origin path: m/84'/0'/0'
# Template: bip84
```

The toolkit also prints each card a second time grouped into 5-character blocks (`ms10e ntrsqqqqqq ...`) — that grouped form is what you punch/engrave.

3. Conventional 2-of-3 multisig card set from 3 seed phrases

Create a 2-of-3 `wsh(sortedmulti(...))` bundle from three seed phrases. Each cosigner gets a card set; the shared **md1** card encodes the wallet policy.

```
$ mnemonic bundle --template wsh-sortedmulti --threshold 2 --network mainnet \  
  --slot "@0.phrase=abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon about" \  
  ↪ \  
  --slot "@1.phrase=legal winner thank year wave sausage worth useful legal winner thank yellow" \  
  --slot "@2.phrase=letter advice cage absurd amount doctor acoustic avoid letter advice cage above"
```

Security note. Putting three live seeds on one machine's command line is shown here only for a self-contained demo. In real multisig you generate each cosigner key on its own (ideally air-gapped) device and combine only the **public** xpubs — no machine ever sees more than one seed. The `argv` form also triggers the toolkit's `secret material on argv` warning; prefer per-device generation.

This emits **3x ms1**, **3x mk1**, and **6x md1** chunks on stdout (the md1 policy is larger, so it spans more chunks). The wallet's identity — the concrete descriptor and first receive address — can be read back from the shared md1 card alone (see §4). For this trio it is:

```
descriptor: wsh(sortedmulti(2,[73c5da0a/87'/0'/0']  
  ↪ xpub661MyMwAqRbcFrooZ2966EcDmVX5MoFXZhuJqXTudvJzwBTBfPQSc5JzX52fvS18oqSdEJXJ4kTGRJ76wPWDUSNJsY5JsgVBQoD6KrbdCLL  
  ↪ /<0;1>/*,[b8688df1/87'/0'/0']  
  ↪ xpub661MyMwAqRbcEnFgxHRLx7i1fnjcBPgc71qy8mVkbGXYukNGMK2XFRbAaCLYEJDUufNoBxTNa68i5MYhqmrEkfhjzgHCUEcvJBhXS5bk4RW  
  ↪ /<0;1>/*,[28645006/87'/0'/0']  
  ↪ xpub661MyMwAqRbcEdy4jr5EtEhQBctfscE6a99DGLr2cW4HnnmBsXDoe3odGzRiw3hcRM5wfKcQmb7s5FjdGrR6SrExXmeopaoY9Lk7tQusDjN  
  ↪ /<0;1>/*))#yjp7hj7w  
first recv: bc1qkssenl2m6t3aynza394sr9m86vt6md2v76kj52jun2xlwrdeaa4q84qtpl
```

4. Card set -> Bitcoin Core wallet descriptor (and how to import)

`mnemonic restore --md1 <chunks...>` reconstructs the watch-only wallet from the **shared md1 card alone** (no seeds needed). Add `--format bitcoin-core` to get a ready-to-import `importdescriptors` payload (external `.../0/*` + change `.../1/*`).

```
$ mnemonic restore --network mainnet \  
  --md1 <md1-chunk-1> --md1 <md1-chunk-2> ... --md1 <md1-chunk-6> \  
  --format bitcoin-core
```

Output (the `importdescriptors` request array):

```
[  
  {  
    "active": true,  
    "desc": "wsh(sortedmulti(2,[73c5da0a/87'/0'/0']  
    ↪ xpub661MyMwAqRbcFrooZ2966EcDmVX5MoFXZhuJqXTudvJzwBTBfPQSc5JzX52fvS18oqSdEJXJ4kTGRJ76wPWDUSNJsY5JsgVBQoD6KrbdCLL  
    ↪ /0/*,[b8688df1/87'/0'/0']  
    ↪ xpub661MyMwAqRbcEnFgxHRLx7i1fnjcBPgc71qy8mVkbGXYukNGMK2XFRbAaCLYEJDUufNoBxTNa68i5MYhqmrEkfhjzghCUEcvJBhXS5bk4RW  
    ↪ /0/*,[28645006/87'/0'/0']  
    ↪ xpub661MyMwAqRbcEdy4jR5EtEhQBctfscE6a99DGLr2cW4HnnmBsXDoe3odGzRiw3hcRM5wfKcQmb7s5FjdGrR6SrExXmeopaoY9Lk7tQusDjN  
    ↪ /0/*))#y65a0dtg",  
    "internal": false,  
    "range": [0, 999],  
    "timestamp": 0  
  },  
  {  
    "active": true,  
    "desc": "wsh(sortedmulti(2,...[same three xpubs].../1/*))#k0gfvz2t",  
    "internal": true,  
    "range": [0, 999],  
    "timestamp": 0  
  }  
]
```

Import into Bitcoin Core. Save the array to `wallet.json`, create a blank descriptor wallet, and load the descriptors:

```
$ mnemonic restore --network mainnet --md1 <chunks...> --format bitcoin-core > wallet.json  
  
$ bitcoin-cli -named createwallet wallet_name="multisig-2of3" \  
  disable_private_keys=true blank=true descriptors=true  
  
$ bitcoin-cli -rpcwallet="multisig-2of3" importdescriptors "$(cat wallet.json)"  
  
$ bitcoin-cli -rpcwallet="multisig-2of3" getnewaddress
```

Tips: `--timestamp` now skips the rescan if the wallet has no prior history (default 0 rescans from genesis); `--range` `↪ 0,4999` widens the gap limit; `--bitcoin-core-version 24` targets older Core. `restore` also emits `--format descriptor` (the bare `wsh(...)#checksum`) for other wallets.

5. Custom degrading-miniscript wallet (distinct keys per tier) + Nunchuk export

A four-tier vault. **Each tier uses its own distinct key set (no key reuse)**, and we deliberately mix all four Bitcoin timelock kinds:

Tier	Spend condition	Timelock kind
1	3-of-3 (K0,K1,K2) + secret word	absolute height – after (1000000)
2	2-of-3 (K3,K4,K5) + secret word	absolute time – after (1893456000)
3	both K6 and K7	relative blocks – older (65535)
4	any 1 of K8,K9,K10	relative time – older (4255898)

That is **11 distinct keys** (3+3+2+3). Absolute locks (**after**) count from the blockchain’s height/clock; relative locks (**older**) count from each coin’s own confirmation.

5.1 Translate the policy to miniscript

Timelock encodings:

- **after**(1000000) – absolute **block height** 1,000,000 (BIP-65; values below 500,000,000 are heights).
- **after**(1893456000) – absolute **Unix time** = **2030-01-01 00:00 UTC** (values at/above 500,000,000 are timestamps).
- **older**(65535) – relative **blocks**: 65,535 blocks (~455 days). (*This is the largest safe relative-block lock; **older** $\hookrightarrow$ (65536) would be BIP-68 consensus-masked to zero, and the toolkit warns if it sees one.*)
- **older**(4255898) – relative **time**: BIP-68’s time flag (bit 22, 0x400000) set, plus 61,594 units x 512 s $\approx$ **365 days**.

The **secret word** is a hashlock shared by tiers 1 and 2 (reusing a *hash* is fine – it is not a key):

```
$ python3 -c "import hashlib; w=b'opensessame'; X=hashlib.sha256(w).digest(); print('preimage X =', X.hex()); print('
  ↳ hash H      =', hashlib.sha256(X).hexdigest())"
preimage X = cd6f70a4440de96063f8dbd7a4a3bbcff6b993af6c83e08d87b359e0760ca9c6
hash H      = a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad
```

The 11 keys here are public BIP-39 test-vector xpubs derived at $m/84'/0'/N'$ for distinct accounts N (3 seeds x several accounts); see the Appendix.

5.2 The guided builder caps complexity – use the raw descriptor path

`mnemonic build-descriptor` runs a full satisfiability + cost preview that it **bounds** for funds-safety. An 11-key, 4-branch policy exceeds that envelope, so the guided builder refuses and points you at the raw `--descriptor` path:

```
$ mnemonic build-descriptor --spec policy.json --network mainnet
build-descriptor: refused -- 1 diagnostic(s):
[over_envelope] root: policy exceeds the always-previewable envelope (2^(11 keys + 2 hashes) x 9 timelock-states >
  ↳ cap 4096); use the raw `--descriptor` path for arbitrarily complex policies
```

(For a policy *within* the envelope – fewer keys – `build-descriptor --spec` validates and emits it for you, the way the bundled archetypes do.) For arbitrarily complex policies you instead hand the miniscript descriptor straight to `bundle / export-wallet`. Validate + canonicalize it first (this also adds the BIP-380 checksum, here `#41d0crra`):

```
$ mnemonic export-wallet --descriptor "$(cat policy.desc)" --format descriptor --network mainnet
```

The descriptor – each tier’s `multi(...)` over its own keys; xpubs abbreviated `xpub6...XXXX` for print (the command emits the full strings):

```
wsh(or_i(and_v(v:after(1000000),and_v(v:sha256(a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad),
  ↳ multi(3,[73c5da0a/84'/0'/0']xpub6Cat...PW6V/<0;1>*,[73c5da0a/84'/0'/1']xpub6Cat...zcUn/<0;1>*,[73c5da0a
  ↳ /84'/0'/2']xpub6Cat...ryy6/<0;1>/*))),or_i(and_v(v:after(1893456000),and_v(v:sha256(
  ↳ a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad),multi(2,[73c5da0a/84'/0'/3']xpub6Cat...PkEC
  ↳ /<0;1>*,[b8688df1/84'/0'/0']xpub6DNf...EgTV/<0;1>*,[b8688df1/84'/0'/1']xpub6DNf...rqpe/<0;1>/*))),or_i(
  ↳ and_v(v:older(65535),multi(2,[b8688df1/84'/0'/2']xpub6DNf...pU51/<0;1>*,[b8688df1/84'/0'/3']xpub6DNf...p3TU
  ↳ /<0;1>/*)),and_v(v:older(4255898),multi(1,[28645006/84'/0'/0']xpub6DBb...ycAm/<0;1>*,[28645006/84'/0'/1']
  ↳ xpub6DBb...hVks/<0;1>*,[28645006/84'/0'/2']xpub6DBb...kBGZ/<0;1>/*))))))#41d0crxa
```

First receive address (Mainnet): bc1q4g7564xxd9hj68hqw5e558cqafhsklerkr0asfzqp6puq74veesrp6qss

5.3 Engrave the card set

Because every key is **distinct**, this *is* a valid BIP-388 wallet policy, so – unlike a key-reusing policy – **bundle** will engrave it. With only public xpubs supplied, the result is **watch-only**:

```
$ mnemonic bundle --descriptor-file policy.desc --network mainnet
```

This emits a watch-only card set: **11x mk1** (one card per key), **24x md1** chunks (the policy is large), and **11x ms1 placeholders** (empty – no secrets, because you supplied xpubs, not seeds). The shared **md1** policy card + the **mk1** set + the **secret word** define the wallet; each signer additionally backs up *their own* seed as a single-sig card set (§2). (If you hold a signer’s seed and want its full ms1 secret card, generate that signer’s bundle from its seed as in §2/§3 – the md1 policy card is identical.)

5.4 Watch-only export for Nunchuk (and other wallets)

There is no dedicated **nunchuk** emitter, but Nunchuk imports miniscript wallets from a **descriptor** or a **BSMS (BIP-129)** record – both of which **export-wallet** produces (and which Bitcoin Core / Sparrow also accept).

Descriptor (Nunchuk: *Add Wallet -> Import -> descriptor*):

```
$ mnemonic export-wallet --descriptor "$(cat policy.desc)" --format descriptor --network mainnet
wsh(or_i(and_v(v:after(1000000),...)))#41d0crxa
note: stdout is watch-only -- public keys only, cannot spend
```

BSMS / BIP-129 (Nunchuk’s multisig import format – carries the descriptor, the */0/**, */1/** derivation, and the first address):

```
$ mnemonic export-wallet --descriptor "$(cat policy.desc)" --format bsms --network mainnet
BSMS 1.0
wsh(or_i(and_v(v:after(1000000),...)))#41d0crxa
/0*/,/1/*
bc1q4g7564xxd9hj68hqw5e558cqafhsklerkr0asfzqp6puq74veesrp6qss
```

After import, fund the address. Each spend path opens only when its lock has matured: tiers 1-2 at an absolute height/date for the whole wallet; tiers 3-4 a fixed span after *each* coin is received. The hashlock tiers (1-2) additionally require revealing the secret word’s 32-byte preimage **x**.

6. Taproot version of the degrading wallet

The §5 vault is `wsh(...)` – one script, so every spend reveals the whole policy. **Taproot** improves on that: it gives a cooperative case a cheap, private **key-path** spend, and splits the fallbacks into **script-tree leaves**, so a spend reveals only the leaf it uses. We keep the same four timelock/hash/multisig tiers as fallbacks and add a cooperative internal key.

Design:

- **Key-path (internal key):** a distinct cooperative key `kint` – spendable anytime, cheapest, and indistinguishable from a single-sig spend on-chain.
- **Script tree (depth-1, two leaves):**
 - Leaf A = tier 1 **or** tier 2 (the two absolute-timelock + secret-word tiers)
 - Leaf B = tier 3 **or** tier 4 (the two relative-timelock tiers)
- **12 distinct keys** (`kint` + the 11 leaf keys). Taproot multisig uses `multi_a`, not `multi`.

A production design would usually MuSig-aggregate the cooperative signers into the key-path; this toolkit places a single designated key there (it does not build MuSig). And `build-descriptor` emits `wsh(...)` only, so – as in §5.2 – taproot goes through the raw `--descriptor` path.

6.1 Why depth-1 (the one-tier-per-leaf limit)

The tidiest layout is one tier per leaf (4 leaves) – but that is a **depth-2** taptree, and the toolkit’s pinned rust-miniscript mis-formats `depth->=2` taptrees (the PR-#953 bug). The toolkit refuses it up front rather than emit a malformed descriptor:

```
$ mnemonic export-wallet --descriptor "tr(Kint,{L1,L2},{L3,L4})" --format descriptor
error: export-wallet script-type derive: taptree branch must have 2 children, but found 1
```

So we use a **depth-1** tree (`<=2` leaves) and pack two tiers per leaf with `or_i`. (A rust-miniscript release `> 13.1.0` containing #953 reopens deep trees – tracked in FOLLOWUP `taproot-coverage-cycle-on-miniscript-gt-13-1-0`.)

6.2 Build + validate

Validate + canonicalize the hand-written `tr(...)` descriptor (checksum `#snerswx7`); xpubs abbreviated `xpub6...XXXX` for print:

```
$ mnemonic export-wallet --descriptor "$(cat taproot.desc)" --format descriptor --network mainnet
tr([73c5da0a/84'/0'/4']xpub6Cat...TuXE/<0;1>/*,{or_i(and_v(v:after(1000000),and_v(v:sha256(
  ↳ a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad),multi_a(3,[73c5da0a/84'/0'/0']xpub6Cat...
  ↳ PW6V/<0;1>/*,[73c5da0a/84'/0'/1']xpub6Cat...zcUn/<0;1>/*,[73c5da0a/84'/0'/2']xpub6Cat...ryy6/<0;1>/*)),and_v
  ↳ (v:after(1893456000),and_v(v:sha256(a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad),multi_a
  ↳ (2,[73c5da0a/84'/0'/3']xpub6Cat...PkEC/<0;1>/*,[b8688df1/84'/0'/0']xpub6DNf...EgTV/<0;1>/*,[b8688df1
  ↳ /84'/0'/1']xpub6DNf...rqpe/<0;1>/*))),or_i(and_v(v:older(65535),multi_a(2,[b8688df1/84'/0'/2']xpub6DNf...
  ↳ pU51/<0;1>/*,[b8688df1/84'/0'/3']xpub6DNf...p3TU/<0;1>/*)),and_v(v:older(4255898),multi_a
  ↳ (1,[28645006/84'/0'/0']xpub6DBb...ycAm/<0;1>/*,[28645006/84'/0'/1']xpub6DBb...hVks
  ↳ /<0;1>/*,[28645006/84'/0'/2']xpub6DBb...kBGZ/<0;1>/*)))#snerswx7
```

`kint ([73c5da0a/84'/0'/4'])` is the key-path; the two `or_i(...)` blocks are the two script leaves. `after(1000000)/after`
↳ `(1893456000)` are the absolute (height / time) locks; `older(65535)/older(4255898)` the relative (blocks / time) locks
– the same four kinds as §5.

6.3 Engrave the card set + first address

Every key is distinct, so it engraves (watch-only):

```
$ mnemonic bundle --descriptor-file taproot.desc --network mainnet
```

-> **12x mk1** (one per key, incl. the internal key), **25x md1** chunks, **12x ms1** placeholders.

First receive address – restore the card set (this round-trip also proves the **real internal key** at the trunk reconstructs, the v0.55.3 non-NUMS feature):

```
$ mnemonic restore --network mainnet --md1 <md1-chunk-1> ... --md1 <md1-chunk-25>
...
first recv: bc1p9stcwz5597fmkxae9343k8edzkcvczf9qp65r6p447pg0et82yqst3d2c
cosigner @0: 73c5da0a [84'/0'/4'] (the key-path internal key)
```

(Restore re-serializes each key as a depth-0 xpub661My... – a different descriptor *string*, identical *addresses*; that is how the md1 card stores keys.)

6.4 Export for wallets (Nunchuk / Core / Sparrow)

descriptor (Nunchuk / Sparrow descriptor import) and bitcoin-core both work for taproot:

```
$ mnemonic export-wallet --descriptor "$(cat taproot.desc)" --format descriptor --network mainnet
tr([73c5da0a/84'/0'/4']xpub6Cat...,{or_i(and_v(v:after(1000000),...))})#snerswx7

$ mnemonic export-wallet --descriptor "$(cat taproot.desc)" --format bitcoin-core --network mainnet
[ {"active":true,"desc":"tr([73c5da0a/84'/0'/4']xpub6Cat.../0/*,{...})#q3s6u2uk","internal":false,"range":[0,999],"
  ↳ timestamp":0}, {"...internal .../1/*...#...":true} ]
```

But **BSMS / BIP-129** is not available for taproot (BIP-129 predates BIP-386 taproot descriptors) – the toolkit says so and points you elsewhere:

```
$ mnemonic export-wallet --descriptor "$(cat taproot.desc)" --format bsms --network mainnet
error: --format bsms does not support taproot (P2trMulti); BIP-129 prerequisites do not yet include BIP-386. Use --
  ↳ format bitcoin-core (Core-importable) or --format sparrow (taproot-capable) ...
```

Spending: the holder of kint spends anytime via the cheap, private key-path; otherwise the same four time-lock/hash/multisig tiers from §5 apply, now as taproot script leaves (a spend reveals only the leaf it uses, not the whole policy).

6.5 Adding a condition: which depth? (a spending-cost comparison)

Suppose you want a fifth spend path: a **new key** K_{new} plus the preimage of a **RIPEMD-160 hashlock** (secret word “please”). Should it be **folded** into an existing leaf (the tree stays depth-1) or given its **own leaf** (forcing depth-2)? Decide on spending cost, not aesthetics.

A taproot script-path witness costs: (satisfaction) + the **revealed leaf script** + a **control block** of $33 + 32 * \text{depth}$ bytes. Two forces oppose each other – folding many conditions into one leaf bloats *every* spend through it (you reveal the unused branches’ opcodes too), while a deeper leaf adds one 32-byte hash per level. A witness byte weighs $1 \text{ WU} = 0.25 \text{ vB}$, so **one extra depth level = +8 vB**.

Hashlock (same two-step scheme as §5): preimage $X = \text{sha256}(\text{"please"})$, descriptor hash $H_p = \text{ripemd160}(X) = 06 \text{ ↳ } d05e2f02fb90ddf98d8cd95d806ba12b27aff4$.

mnemonic compare-cost reports per-condition witness vbytes (key-agnostic – abstract labels A,B,... are auto-dummy-keyed). Spend the new condition $F + \text{preimage}$:

Folded – the new tier joins Leaf B, so spending it reveals all of $t_3+t_4+t_5$:

```
$ mnemonic compare-cost --miniscript "or_i(and_v(v:older(65535),multi(2,A,B)),or_i(and_v(v:older(4255898),multi(1,C,D
  ↳ ,E)),and_v(v:pk(F),ripemd160(06d05e2f02fb90ddf98d8cd95d806ba12b27aff4))))"

Condition          | wsh vB | tr vB | ...
-----+-----+-----+-----
A + B + older(blocks) |    79 |   147 | ...
C + older(512s)      |    61 |   132 | ...
D + older(512s)      |    61 |   132 | ...
E + older(512s)      |    61 |   132 | ...
F + preimage(h0)     |    69 |   139 | ...   <- reveals the whole 3-tier leaf
```

Dedicated – the new tier is its own leaf, revealing only itself:

```
$ mnemonic compare-cost --miniscript "and_v(v:pk(F),ripemd160(06d05e2f02fb90ddf98d8cd95d806ba12b27aff4))"
Condition          | wsh vB | tr vB | ...
-----+-----+-----+-----
```

F + preimage(h0)	68	90	...	<- reveals only t5
------------------	----	----	-----	--------------------

`compare-cost` models each input as a single leaf at the tree root (depth-0, 33-byte control block). Add the real Merkle depth (+8 vB per level):

Placement	revealed script	tr vB (depth-0)	real depth	+ control block	real tr
Folded into	whole	139	1 (still 2	+8	~147 vB
Leaf B	<code>or_i(t3,t4,t5)</code>		leaves)		
Dedicated leaf	just t5	90	2 (now 3	+16	~106 vB
			leaves)		

A dedicated leaf is ~41 vB (~28%) cheaper to spend the new condition: not revealing t3+t4 (~49 vB) beats the +8 vB for the extra depth level. And it is better still in practice, because (a) folding also makes *t3 and t4* spends reveal t5's script, and (b) a 3-leaf tree can place the **most-used** condition in the shallow depth-1 slot (`{t5,{A,B}}`) and bury cold paths deeper.

Rule of thumb: in taproot, one-condition-per-leaf almost always wins – each sibling script you avoid revealing is worth far more than the 8 vB/level of depth – and you order leaves hot-shallow / cold-deep.

The catch: cost says depth-2, but the toolkit cannot build depth->=2 yet (the PR-#953 `taptree-Display` bug, §6.1) – so today the only buildable option is the folded depth-1 leaf at the ~28% premium. That premium is the concrete, quantified motive for landing the upstream fix (`taproot-coverage-cycle-on-miniscript-gt-13-1-0`).

Appendix — the public test seeds used

Slot	BIP-39 test phrase	Fingerprint
A / @0	abandon abandon ... about	73c5da0a
B / @1	legal winner thank year wave sausage	b8688df1
	↪ worth useful legal winner thank	
	↪ yellow	
C / @2	letter advice cage absurd amount	28645006
	↪ doctor acoustic avoid letter	
	↪ advice cage above	

These are world-known BIP-39 vectors with no funds. Generated with `mnemonic v0.55.3` on Linux. See the in-repo manual (`docs/manual/`) for the authoritative per-flag reference.